

IMPLEMENTATION GUIDE

Paper Vendo Machine Capstone Refactor

Machine flow, database contract, web reporting, wiring additions, and deployment checks

Prepared	Scope	Status
2 August 2026	Mega, ESP32, Supabase, PERN website	Implementation refactor; hardware validation required

Release rule. The machine verifies and records the full change payment before it receives a dispense plan. It never relies on a manual D button or a website calculation.

What changed

- A complete cart is reserved atomically before any motor moves, including products, user units, physical output quantity, stock, and exact change.
- The Mega returns verified change first through a hopper sensor. Only CHANGE_OK allows the ESP32 to request the dispense plan.
- Transaction lines store units_requested separately from qty_requested and qty_dispensed. For paper, units are the customer choice and quantity is physical sheets.
- The website reads stored units and price snapshots; it no longer reconstructs units from amount, sheet settings, or quantities after the sale.
- Four paper channels are physical feeders by size. Budget and Standard are logical catalog products that may share a feeder only when their stock is physically identical.

Final transaction flow

1. Customer inserts coins. The Mega maintains credit in pesos and sends exact cents only when checkout begins.
2. Customer selects Paper, then Budget or Standard, then one of four sizes; or selects one of three pen channels. The cart contains requested units, not calculated sheets.
3. Mega inhibits the coin acceptor and sends one RESERVE message for the whole cart. ESP32 calls machine_reserve_transaction, which validates price, units, stock, shared feeder capacity, and exact change in one database transaction.
4. ESP32 returns RESERVED with transaction ID, subtotal, and change due. Mega drives the change hopper and counts each exit-sensor edge.
5. Only after every change coin is sensor-confirmed does Mega send CHANGE_OK. ESP32 marks CHANGE_PAID and returns the physical PLAN.
6. Mega dispenses each plan item. Paper needs an exit-sensor cycle for each physical sheet; pens need the IR drop confirmation. Mega sends actual quantities in FINISH.
7. ESP32 calls machine_finish_transaction. The database commits inventory consumption, marks lines DISPENSED or FAILED, and exposes stored results to the website.

Failure behavior. If exact change cannot be confirmed, Mega sends CHANGE_FAIL and the reservation is cancelled. If change was confirmed but a dispenser sensor fails, the transaction is stored as FAILED_DISPENSE with actual output; this requires operator resolution.

Data model and calculation rule

Concept	Stored field	Meaning
Customer choice	units_requested	How many sellable units the user selected.
Paper physical output	qty_requested	units_requested x sheets_per_unit_snapshot.
Confirmed output	qty_dispensed	Sensor-confirmed sheets or pens actually released.
Price evidence	unit_price_cents	Price at reservation; later setting edits cannot rewrite history.
Money	*_cents	Integer cent values avoid floating-point money errors.

Formula: subtotal_cents = sum(unit_price_cents x units_requested). change_due_cents = credit_received_cents - subtotal_cents. For paper, qty_requested = units_requested x sheets_per_unit_snapshot.

Wiring additions only

Existing pen steppers, pen IR sensors, TFT, coin acceptor, OLED, servos, and Mega-ESP UART remain connected as they are. Add only the following paths:

Hardware	Mega pins	Purpose
Paper drivers 1 to 4	STEP/DIR: 32/33, 34/35, 36/37, 38/39; shared EN: 40	Drive one adjustable paper feeder per physical size.
Paper exit sensors 1 to 4	41, 42, 43, 44	Confirm every sheet after it leaves its channel.
Change hopper motor driver	14	Releases reserved P1 change coins.
Change hopper coin sensor	15	Confirms every released change coin before dispensing.
Coin acceptor inhibit	16	Freezes coin acceptance after checkout starts so the credit snapshot cannot change mid-transaction.
Mega to ESP UART	Mega Serial1 18/19 to ESP Serial2 16/17 through level shifting	Carries reservation and result protocol; protect ESP 3.3 V RX.

Mechanical constraint. Four paper compartments support eight catalog choices only when Budget and Standard for the same size use the same physical paper. If brands differ in stock, color, weight, or finish, use eight physical feeders.

File-by-file implementation summary

Key line ranges identify the main implementation areas, not every supporting line.

File	Key lines	Specific change and purpose
Cloud_Paper_Vendo.sql	1-328; 152-326	Clean development schema: channels, exact change, transaction header/lines, atomic reserve/change/finish RPCs. Purpose: authoritative, auditable data flow.

File	Key lines	Specific change and purpose
updated_arduino_mega.ino	20-36; 178-226; 571-693; 949-1258	Pins, coin inhibit, brand selection, display price, cart reservation, verified change, sensor routines, FINISH results. Purpose: stable credit snapshot, change before dispense, and machine confirmation.
production_esp32.ino	1-229; 125-212	Serial protocol and Supabase RPC gateway. Purpose: machine-to-database state changes.
backend/routes/machine.js	1-276; 102-257	Stored units/output, inventory updates, completed-line analytics. Purpose: stop frontend inference.
Transactions.jsx	1-72; 59-60	Stored units, expected/actual output, snapshots, statuses. Purpose: auditability.
Reports.jsx	56-145; 319-340	Uses units_requested and product snapshots. Purpose: accurate historical reports.
backend/routes/auth.js	30-42	Removes plaintext password fallback. Purpose: bcrypt-only login.

Deployment and calibration checklist

- Back up real Supabase data. Cloud_Paper_Vendo.sql is a clean-reset development script and drops the listed tables.
- Run SQL on the development database, then enter real stock, prices, sheets per unit, and hopper coin count.
- Configure real Wi-Fi and Supabase values in production_esp32.ino. Do not place credentials in source control.
- Install and calibrate four paper modules: driver direction, step timing, separator, and exit sensor. A paper sheet counts only on clear -> blocked -> clear.
- Install a motor driver between Mega D14 and the hopper motor; never drive the motor directly from a Mega pin. Adjust active level for your module.
- Run full-path tests: pen, multi-sheet paper, mixed cart, zero change, valid change, no exact change, sensor failure, and Wi-Fi loss before and after change payment.

Verification completed in this refactor

- Backend JavaScript syntax checked with Node.
- Frontend production build completed successfully with Vite.
- Mega sketch structure checked for balanced braces. Physical Arduino compile and hardware tests remain required because this workspace has no Arduino CLI or hardware access.
- This PDF was rendered and visually inspected before delivery.

Operator acceptance criteria

- No good is released unless database reservation succeeds and the change sensor verifies the change due.
- A website transaction shows selected units, actual physical output, price snapshot, status, and timestamp without recomputing them.
- Paper inventory reduces by confirmed physical sheets; pen inventory reduces by confirmed pens.
- A sensor-short dispense is visible as a failed line and failed transaction, rather than silently marked successful.